Order Flow Imbalance - A High Frequency Trading Signal

Feb 2, 2022

I'll show you how to calculate the 'order flow imbalance' and build a high-frequency trading signal with the results. We'll see if it is a profitable strategy and how it might be useful as a market indicator.

Enjoy these types of posts? Then you should sign up for my newsletter.

Type your email... Subscribe

By subscribing you agree to Substack's Terms of Use, our Privacy Policy and our Information collection notice

substack

A couple of months ago I attended the Oxford Math Finance seminar where there was a presentation on order book dynamics to predict short-term price direction. Nicholas Westray presented Deep Order Flow Imbalance: Extracting Alpha at Multiple Horizons from the Limit Order Book. By using deep learning they predict future price movements using common neural network architectures such as the basic multi-layer perceptron (MLP), Long Term Short Memory network (LSTM) and convolutional neural networks (CNN). Combining all three networks types lets you extract the strengths of each network:

- CNNs: Reduce frequency variations.
  - The variations between each input reduce to some common factors.
- LSTMs: Learn temporal structures
  - How are the different inputs correlated with each other such as autocorrelation structure.
- MPLs: Universal approximators.
  - An MLP can approximate any function.

A good reference for LSTM-CNN combinations is DeepLOB: Deep Convolution Neural Networks for Limit Order Books where they use this type of neural network to predict whether the market is moving up or down.

The **Deep Order Flow** talk was an excellent overview of deep learning concepts and described how there is a great overlap between computer vision and the state of the order book. You can build an "image" out of the different order book levels and pass this through the neural networks. My main takeaway from the talk was the concept of *Order Flow Imbalance*. This is a transformation that uses the order book to build a feature to predict future returns.

I'll show you how to calculate the order flow imbalance and see how well it predicts future returns.

The Setup

I have a QuestDB database with the best bid and offer price and size at those levels for BTCUSD from Coinbase over roughly 24 hours. To read how I collected this data check out my previous post on streaming data into QuestDB.

Julia easily connects to QuestDB using the `LibPQ.jl` package. I also load in the basic data manipulation packages and some statistics modules to calculate the necessary values.

```
using LibPQ
using DataFrames, DataFramesMeta
using Plots
using Statistics, StatsBase
using CategoricalArrays
using Dates
using RollingFunctions

conn = LibPQ.Connection("""
             dbname=qdb
             host=127.0.0.1
             password=quest
             port=8812
             user=admin""")
```

Order flow imbalance is about the changing state of the order book. I need to pull out the full best bid best offer table. Each row in this table represents when the best price or size at the best price changed.

```
bbo = execute(conn,
    "SELECT *
     FROM coinbase_bbo") |> DataFrame
dropmissing!(bbo);
```

I add the mid-price in too, as we will need it later.

```
bbo = @transform(bbo, mid = (:ask .+ :bid) / 2);
```

It is a big dataframe, but thankfully I've got enough RAM.

Calculating Order Flow Imbalance

Order flow imbalance represents the changes in supply and demand. With each row one of the price or size at the best bid or ask changes which corresponds to change in the supply or demand, even at a high frequency level, of Bitcoin.

- Best bid or size at the best bid *increase* -> *increase* in demand.
- Best bid or size at the best bid *decreases* -> *decrease* in demand.
- Best ask *decreases* or size at the best ask *increases* -> *increase* in supply.
- Best ask *increases* or size at the best ask *decreases* -> *decrease* in supply.

Mathematically we summarise these four effects at from time $n-1$ to n as:

$$e_n = I_{\{P_n^B \geq P_{n-1}^B\}} q_n^B - I_{\{} P_n^B \leq P_{n-1}^B \} q_{n-1}^B - I_{\{} P_n^A \leq P_{n-1}^A \} q_n^A + I_{\{} P_n^A \geq P_{n-1}^A \} q_{n-1}^A,$$

where P is the best price at the bid (P^B) or ask (P^A) and q is the size at those prices.

Which might be a bit easier to read as Julia code:

```
e = Array{Float64}(undef, nrow(bbo))
fill!(e, 0)
```

```
for n in 2:nrow(bbo)

    e[n] = (bbo.bid[n] >= bbo.bid[n-1]) * bbo.bidsize[n] -
    (bbo.bid[n] <= bbo.bid[n-1]) * bbo.bidsize[n-1] -
    (bbo.ask[n] <= bbo.ask[n-1]) * bbo.asksize[n] +
    (bbo.ask[n] >= bbo.ask[n-1]) * bbo.asksize[n-1]

end

bbo[!, :e] = e;
```

To produce an Order Flow Imbalance (OFI) value, you need to aggregate e over some time-bucket. As this is a high-frequency problem I'm choosing 1 second. We also add in the open and close price of the buckets and the return across this bucket.

```
bbo = @transform(bbo, timestampfloor = floor.(:timestamp, Second(1)))
bbo_g = groupby(bbo, :timestampfloor)
modeldata = @combine(bbo_g, ofi = sum(:e), OpenPrice = first(:mid), ClosePrice =
modeldata = @transform(modeldata, OpenCloseReturn = 1e4*(log.(:ClosePrice) .- log
modeldata = modeldata[2:(end-1), :]
first(modeldata, 5)
```

5 rows × 6 columns

	timestampfloor	ofi	OpenPrice	ClosePrice	NTicks	OpenCloseReturn
	DateTime	Float64	Float64	Float64	Int64	Float64
1	2021-07-24T08:50:36	0.0753159	33655.1	33655.1	77	0.0
2	2021-07-24T08:50:37	4.44089e-16	33655.1	33655.1	47	0.0
3	2021-07-24T08:50:38	0.0	33655.1	33655.1	20	0.0
4	2021-07-24T08:50:39	3.05727	33655.1	33655.1	164	0.0
5	2021-07-24T08:50:40	2.40417	33655.1	33657.4	278	0.674467

Now we do the usual train/test split by selecting the first 70% of the data.

```
trainInds = collect(1:Int(floor(nrow(modeldata)*0.7)))
trainData = modeldata[trainInds, :]
testData = modeldata[Not(trainInds), :];
```

We are going to fit a basic linear regression using the OFI value as the single predictor.

```
using GLM
```

```
ofiModel = lm(@formula(OpenCloseReturn ~ ofi), trainData)
```

```
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.Der

OpenCloseReturn ~ 1 + ofi

Coefficients:
──────────────────────────────────────────────────────────────────────────────
                  Coef.   Std. Error       t  Pr(>|t|)  Lower 95%   Upper 95%
──────────────────────────────────────────────────────────────────────────────
(Intercept)  -0.0181293  0.00231571    -7.83    <1e-14  -0.022668  -0.0135905
ofi           0.15439    0.000695685  221.92    <1e-99   0.153026   0.155753
──────────────────────────────────────────────────────────────────────────────
```

We see a positive coefficient of 0.15 which is very significant.

```
r2(ofiModel)
```

```
0.3972317963590547
```

A very high in-sample R^2.

```
predsTrain = predict(ofiModel, trainData)
predsTest = predict(ofiModel, testData)

(mean(abs.(trainData.OpenCloseReturn .- predsTrain)),
    mean(abs.(testData.OpenCloseReturn .- predsTest)))
```

```
(0.3490577385082666, 0.35318460250890665)
```

Comparable mean absolute error (MAE) across both train and test sets.

```
sst = sum((testData.OpenCloseReturn .- mean(testData.OpenCloseReturn)) .^2)
ssr = sum((predsTest .- mean(testData.OpenCloseReturn)) .^2)
ssr/sst
```

```
0.4104873667550974
```

An even better R^2 in the test data

```
extrema.([predsTest, testData.OpenCloseReturn])
```

```
2-element Vector{Tuple{Float64, Float64}}:
 (-5.400295917229609, 5.285718311926791)
 (-11.602503514716034, 11.46049286770534)
```

But doesn't quite predict the largest or smallest values.

So overall:

- Comparable R2 and MAE values across the training and test sets.

- Positive coefficient indicates that values with high positive order flow imbalance will have a large positive return.

But, this all suffers from the cardinal sin of backtesting, we are using information from the future (the sum of the e values to form the OFI) to predict the past. By the time we know the OFI value, the close value has already happened! We need to be smarter if we want to make trading decisions based on this variable.

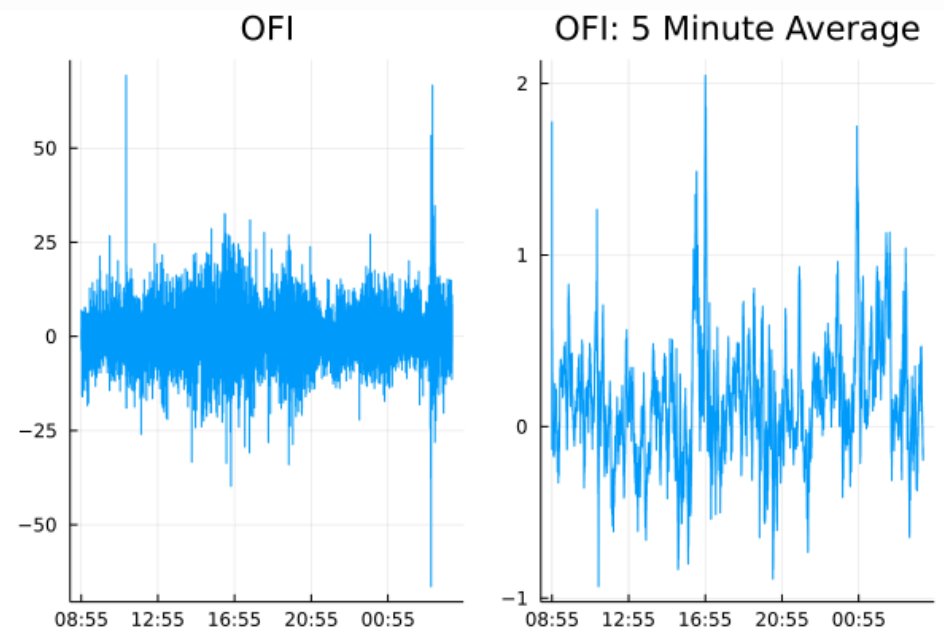
So whilst it doesn't give us an actionable signal, we know that it can explain price moves, we know just have to reformulate our model and make sure there is no information leakage.

Building a Predictive Trading Signal

I now want to see if OFI can be used to predict future price returns. First up, what do the OFI values look like and what about if we take a rolling average?

Using the excellent `RollingFunctions.jl` package we can calculate the five-minute rolling average and compare it to the raw values.

```
xticks = collect(minimum(trainData.timestampfloor):Hour(4):maximum(trainData.time
xtickslabels = Dates.format.(xticks, dateformat"HH:MM")

ofiPlot = plot(trainData.timestampfloor, trainData.ofi, label = :none, title="OFI
ofi5minPlot = plot(trainData.timestampfloor, runmean(trainData.ofi, 60*5), title=
plot(ofiPlot, ofi5minPlot, fmt=:png)
```



It's very spiky, but taking the rolling average smooths it out. To scale the OFI values to a known range, I'll perform the Z-score transform using the rolling five-minute window of both the mean and variance. We will also use the close to close returns rather than the open-close returns of the previous model and make sure it is lagged correctly to prevent information leakage.

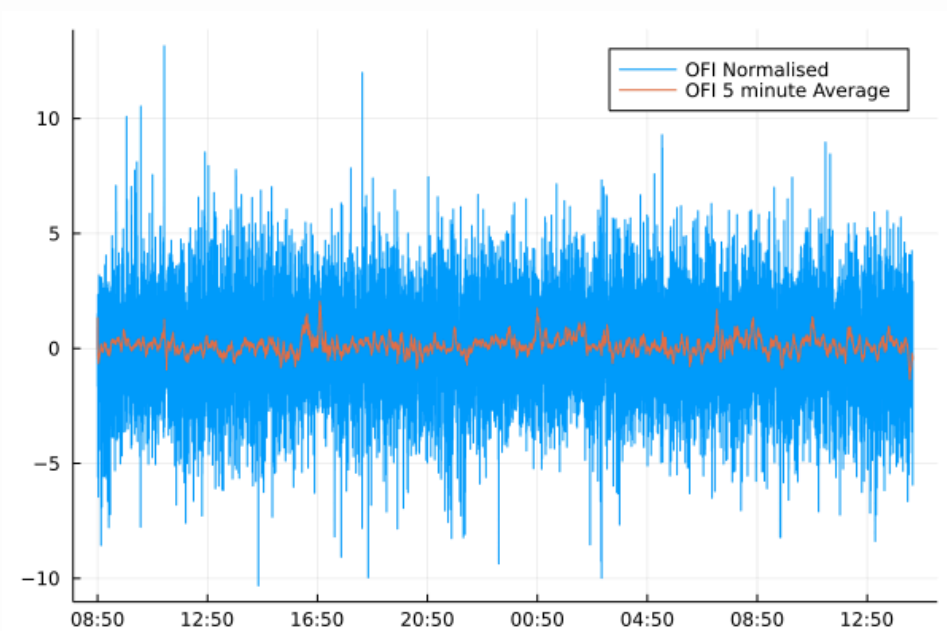
```
modeldata = @transform(modeldata, ofi_5min_avg = runmean(:ofi, 60*5),
                                  ofi_5min_var = runvar(:ofi, 60*5),
                                  CloseCloseReturn = 1e4*[diff(log.(:ClosePrice)
```

```
modeldata = @transform(modeldata, ofi_norm = (:ofi .- :ofi_5min_avg) ./ sqrt.(:of

modeldata[!, :CloseCloseReturnLag] = [NaN; modeldata.CloseCloseReturn[1:(end-1)]]

modeldata[1:7, [:ofi, :ofi_5min_avg, :ofi_5min_var, :ofi_norm, :OpenPrice, :Close
```

7 rows × 7 columns

	ofi	ofi_5min_avg	ofi_5min_var	ofi_norm	OpenPrice	ClosePrice	CloseClose
	Float64	Float64	Float64	Float64	Float64	Float64	Float64
1	0.0753159	0.0753159	0.0	NaN	33655.1	33655.1	0.0
2	4.44089e-16	0.037658	0.00283625	-0.707107	33655.1	33655.1	0.0
3	0.0	0.0251053	0.00189083	-0.57735	33655.1	33655.1	0.0
4	3.05727	0.783146	2.29977	1.49959	33655.1	33655.1	0.674467
5	2.40417	1.10735	2.25037	0.864473	33655.1	33657.4	1.97263
6	2.4536	1.33172	2.10236	0.773732	33657.4	33664.0	0.252492
7	-2.33314	0.808173	3.67071	-1.63959	33664.0	33664.9	-0.531726

```
xticks = collect(minimum(modeldata.timestampfloor):Hour(4):maximum(modeldata.time
xtickslabels = Dates.format.(xticks, dateformat"HH:MM")

plot(modeldata.timestampfloor, modeldata.ofi_norm, label = "OFI Normalised", xtic
plot!(modeldata.timestampfloor, modeldata.ofi_5min_avg, label="OFI 5 minute Avera
```



The OFI values have been compressed from $(-50, 50)$ to $(-10, 10)$. From the average values we can see periods of positive and negative regimes.

When building the model we split the data into a training and testing sample, throwing away the early values where the was not enough values for the rolling statistics to calculate.

We use a basic linear regression with just the normalised OFI value.

```
trainData = modeldata[(60*5):70000, :]
testData = modeldata[70001:(end-1), :]

ofiModel_predict = lm(@formula(CloseCloseReturn ~ ofi_norm), trainData)
```

```
StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.Der

CloseCloseReturn ~ 1 + ofi_norm

Coefficients:
──────────────────────────────────────────────────────────────────────────────
                 Coef.  Std. Error      t  Pr(>|t|)    Lower 95%   Upper 95%
──────────────────────────────────────────────────────────────────────────────
(Intercept)  0.0020086  0.00297527   0.68    0.4996  -0.00382293  0.00784014
ofi_norm     0.144358   0.00292666  49.33    <1e-99   0.138622    0.150094
──────────────────────────────────────────────────────────────────────────────
```

A similar value in the coefficient compared to our previous model and it remains statistically significant.

```
r2(ofiModel_predict)
```

```
0.033729601695801414
```

Unsurprisingly, a massive reduction on in-sample R^2. A value of 3% is not that bad, in the Deep Order Flow paper they achieve values of around 1% but over a much larger dataset and across multiple stocks. My 24 hours of Bitcoin data is much easier to predict.

```
returnPredictions = predict(ofiModel_predict, testData)

testData[!, :CloseClosePred] = returnPredictions

sst = sum((testData.CloseCloseReturn .- mean(testData.CloseCloseReturn)) .^2)
ssr = sum((testData.CloseClosePred .- mean(testData.CloseCloseReturn)) .^2)
ssr/sst
```

```
0.030495583445248473
```

The out-of-sample R^2 is also around 3%, so not that bad really in terms of overfitting. It looks like we've got a potential model on our hands.

Does This Signal Make Money?

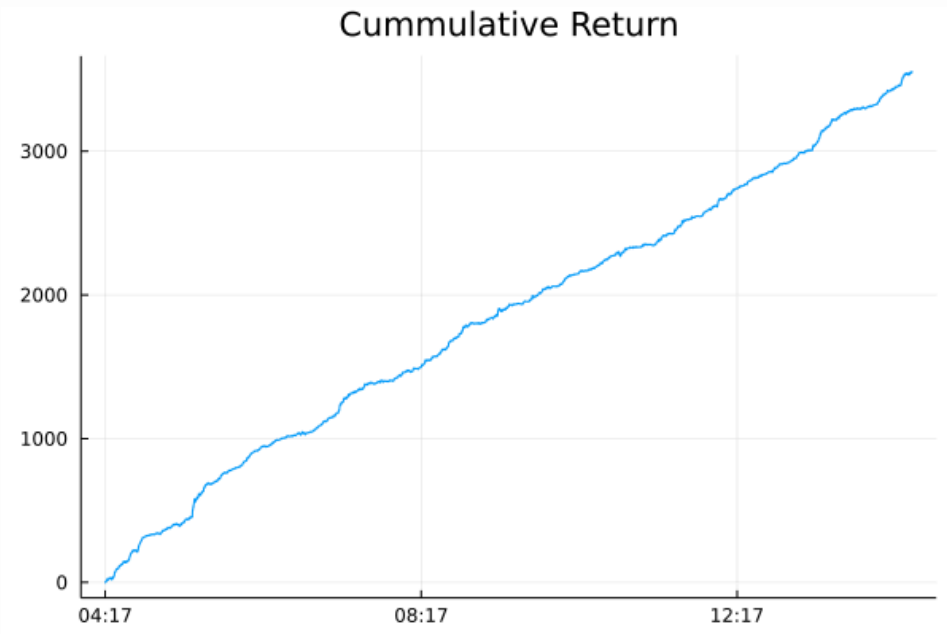
We can now go through a very basic backtest to see if this signal is profitable to trade. This will all be done in pure Julia, without any other packages.

Firstly, what happens if we go long every time the model predicts a positive return and likewise go short if the model predicts a negative return. This means simply taking the sign of the model prediction and

multiplying it by the observed returns will give us the returns of the strategy.

In short, this means if our model were to predict a positive return for the next second, we would immediately buy at the close and be filled at the closing price. We would then close out our position after the second elapsed, again, getting filled at the next close to produce a return.

```
xticks = collect(minimum(testData.timestampfloor):Hour(4):maximum(testData.timest
xtickslabels = Dates.format.(xticks, dateformat"HH:MM")

plot(testData.timestampfloor, cumsum(sign.(testData.CloseClosePred) .* testData.(
    label=:none, title = "Cummulative Return", fmt=:png, xticks = (xticks, xticks
```



Up and to the right as we would hope. So following this strategy would make you money. Theoretically. But is it a *good* strategy? To measure this we can calculate the Sharpe ratio, which is measuring the overall profile of the returns compared to the volatility of the returns.

```
moneyReturns = sign.(testData.CloseClosePred) .* testData.CloseCloseReturn
mean(moneyReturns) ./ std(moneyReturns)
```

```
0.11599938576235787
```

A Sharpe ratio of 0.12 if we are generous and round up. Anyone with some experience in these strategies is probably having a good chuckle right now, this value is terrible. At the very minimum, you would like a value of 1, i.e. that your average return is greater than the variance in returns, otherwise you are just looking at noise.

How many times did we correctly guess the direction of the market though? This is the hit ratio of the strategy.

```
mean(abs.((sign.(testData.CloseClosePred) .* sign.(testData.CloseCloseReturn))))
```

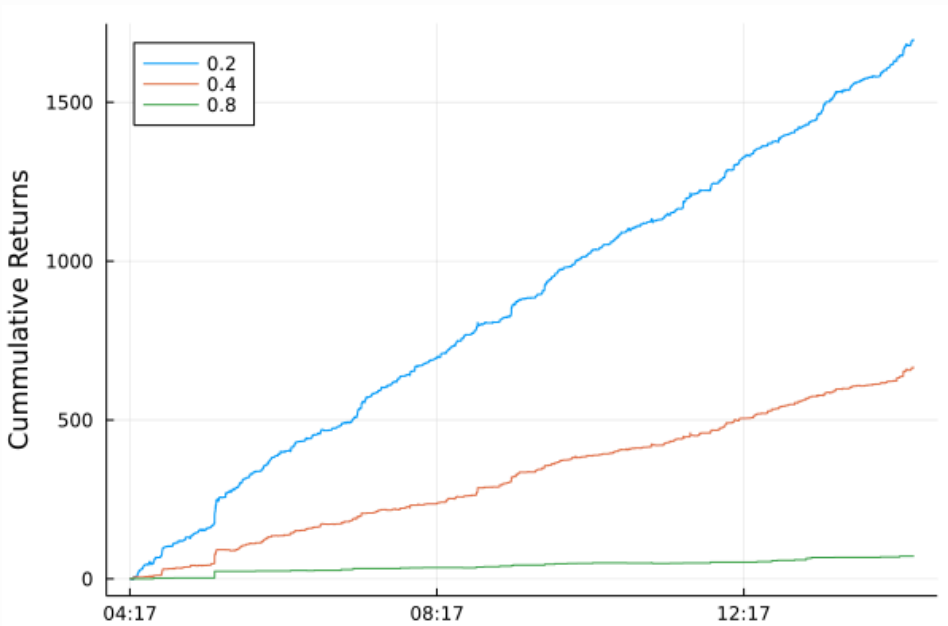
```
0.530163738236414
```

So 53% of the time I was correct. 3% better than a coin toss, which is good and shows there is a little bit of information in the OFI values when predicting.

Does a Threshold Help?

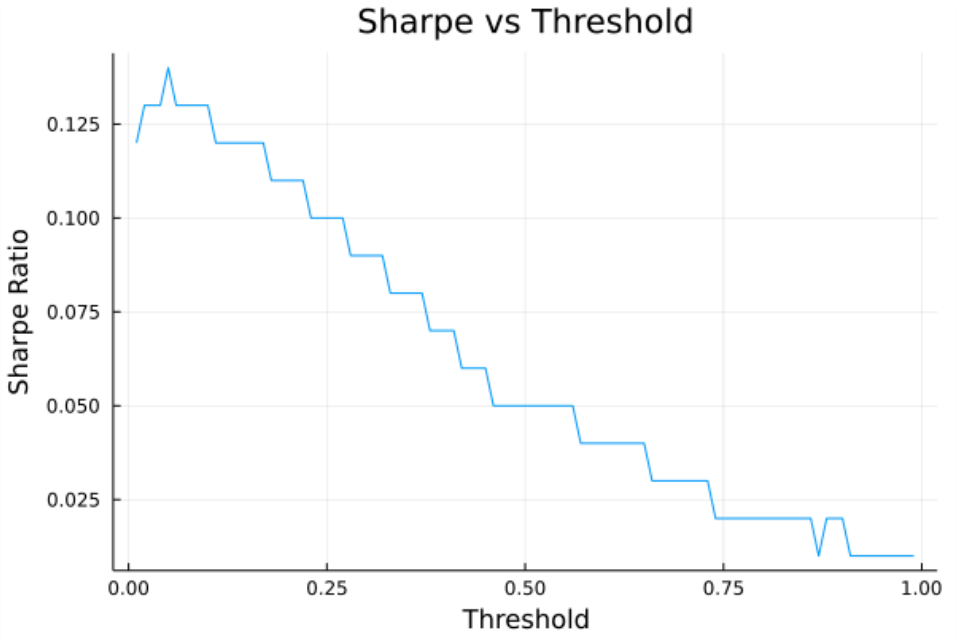
Should we be more selective when we trade? What if we set a threshold and only trade when our prediction is greater than that value. Plus the same in the other direction. We can iterate through lots of potential thresholds and see where the Sharpe ratios end up.

```
p = plot(ylabel = "Cummulative Returns", legend=:topleft, fmt=:png)
sharpes = []
hitratio = []

for thresh in 0.01:0.01:0.99
  trades = sign.(testData.CloseClosePred) .* (abs.(testData.CloseClosePred) .> th

  newMoneyReturns = trades .* testData.CloseCloseReturn

  sharpe = round(mean(newMoneyReturns) ./ std(newMoneyReturns), digits=2)
  hr = mean(abs.((sign.(trades) .* sign.(testData.CloseCloseReturn))))

  if mod(thresh, 0.2) == 0
    plot!(p, testData.timestampfloor, cumsum(newMoneyReturns), label="$(thresh)",
  end
  push!(sharpes, sharpe)
  push!(hitratio, hr)
end
p
```



The equity curves look worse with each higher threshold.

```
plot(0.01:0.01:0.99, sharpes, label=:none, title = "Sharpe vs Threshold", xlabel
```



A brief increase in Sharpe ratio if we set a small threshold, but overall, steadily decreasing Sharpe ratios once we start trading less. For such a simple and linear model this isn't surprising, but once you start chucking more variables and different modeling approaches into the mix it can shed some light on what happens around the different values.

Why you shouldn't trade this model

So at the first glance, the OFI signal looks like a profitable strategy. Now I will highlight why it isn't in practice.

- Trading costs will eat you alive

I've not taken into account any slippage, probability of fill, or anything that a real-world trading model would need to be practical. As our analysis around the Sharpe ratio has shown, it wants to trade as much as possible, which means transaction costs will just destroy the return profile. With every trade, you will pay the full bid-ask spread in a round trip to open and then close the trade.

- The Sharpe ratio is terrible

With a Sharpe ratio < 1 shows that there is not much actual information in the trading pattern, it is getting lucky vs the actual volatility in the market. Now, Sharpe ratios can get funky when we are looking at such high-frequency data, hence why this bullet point is second to the trading costs.

- It has been trained on a tiny amount of data.

Needless to say, given that we are looking at seconds this dataset could be much bigger and would give us greater confidence in the actual results once expanded to a wider time frame of multiple days.

- I've probably missed something that blows this out of the water

Like everything I do, there is a strong possibility I've gone wrong somewhere, forgotten a minus, ordered a time-series wrong, and various other errors.

How this model might be useful

- An overlay for a market-making algorithm

Making markets is about posting quotes where they will get filled and collecting the bid-ask spread. Therefore, because our model appears to be able to predict the direction fairly ok, you could use it to place a quote where the market will be in one second, rather than where it is now. This helps put your quote at the top of the queue if the market does move in that direction. Secondly, if you are traded with and need to hedge the position, you have an idea of how long to wait to hedge. If the market is moving in your favour, then you can wait an extra second to hedge and benefit from the move. Likewise, if this model is predicting a move against your inventory position, then you know to start aggressively quoting to minimise that move against.

- An execution algorithm

If you are potentially trading a large amount of bitcoin, then you want to split your order up into lots of little orders. Using this model you then know how aggressive or passive you should trade based on where the market is predicted to move second by second. If the order flow imbalance is trending positive, the market is going to go up, so you want to increase your buying as not to miss out on the move and again, if the market is predicted to move down, you'll want to slow down your buying so that you fully benefit from the lull.

Conclusion

Overall hopefully you now know more about order flow imbalance and how it can somewhat explain returns. It also has some predictive power and we use that to try and build a trading strategy around the signal.

We find that the Sharpe ratio of said strategy is poor and that overall, using it as a trading signal on its own will not have you retiring to the Bahamas.

This post has been another look at high-frequency finance and the trials and tribulations around this type of data.

ALSO ON **DM13450**

Fixture Difficulty and Fantasy Premier ...

4 years ago · 1 comment

In my last blog post I showed how the odds can be used to measure the ...

Benchmarking maps, loops, generators ...

7 years ago · 4 comments

A few weeks ago I wrote a blog post about the speed differences between map ...

State of the Market - Infinite State Hidden ...

6 years ago · 5 comments

My dirichletprocess package for R has the ability to fit Infinite Hidden Markov ...

Palmer Peng Introduction

5 years ago · 2 c

Dirichlet proce complicated b post will be sir

8 Comments

Login ▾

G

Join the discussion…

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name

Share

Best Newest Oldest

F

flyingpig

4 years ago

The sharp ratio seems to be high if annualized. Trading cost is much more critical

1 0 Reply

J

jjh

4 years ago

Hi Dean,

Great work. Do you algorithmically trade your own portfolio? If so, do you use Julia for that, or have you simply been taking a liking to Julia for pleasure? I am curious to know if you have any opinions on using Julia to build such an infrastructure, or if there are some weaknesses you think prevent it from being optimal for such an application.

Thanks

1 0 Reply

D

Dean Markwick Mod → jjh

4 years ago

I've never explored that, but have been asked about it multiple times, so might have to take the plunge and give it a try.

1 0 Reply

P

Pankaj Chaudhary

3 years ago edited

Hi Dean ,

Appreciate your work !

I have doubt in the formulae given in the paper equation (2 & 3) not matching with your mathematically equation for OFI.

As per equation (2) bid(n) < bid(n-1) then (-q bid(n)) not (q bid(n-1)) same for Ask also as per equation (3) in the paper

Thanks !

0 0 Reply

P

Pankaj Chaudhary → Pankaj Chaudhary

3 years ago

I think , my question was invalid because you used the equation (in the blog) for Level 1 data.

0 0 Reply

Eric Jia
3 years ago
OpenCloseReturn = 1e4*(log.(:ClosePrice) .- log.(:OpenPrice))) to (ClosePrice - OpenPrice)?

0 0 Reply

Shane
4 years ago
Looks like you need to annualize your sharpe ratio.

0 0 Reply

Dean Markwick Mod → Shane
4 years ago
Yes, mucked up the calculation there, correcting it soon

0 0 Reply

Subscribe Privacy Do Not Sell My Data

Dean Markwick

Personal website for Dean Markwick. If you like stats, sports and rambling, you've come to the right place. All rights reserved.

dm13450
deanmarkwick13450
deanmarkwick
rss

Type your email... Subscribe

By subscribing you agree to Substack's Terms of Use, our Privacy Policy and our Information collection notice

substack